

FileLab — Evaluacion practica: repositorios de Personas

Duracion: 1 hora y 45 minutos (105 minutos)

Modalidad: Individual

Puntuacion maxima: 10 puntos (9 de funcionalidad + 1 de JavaDoc condicionado)

Tecnologias: Java 21, Maven, JUnit 5, Apache Commons CSV y Jackson (JSON/XML).

1. Situacion y objetivo

Una academia almacena informacion sobre personas en tres formatos: **CSV, JSON y XML**, y utiliza un archivo **.properties** para configurar su aplicacion. Tu tarea es completar una serie de componentes especificos para la entidad **Persona** y comprobar su funcionamiento con los archivos ya existentes.

No debes crear ni programar un CRUD desde cero. Se entregan resueltos **IRepository<T, ID>** y **AbstractFileRepository<T, ID>**, asi como los ejemplos de apoyo **CrudDemoCsv**, **CrudDemoJson** y **CrudDemoXml** en una carpeta independiente. **Estas implementaciones CRUD no se evaluan** y no deben modificarse. Lo evaluable es la adaptacion de la persistencia a **Persona**, las consultas especificas y la gestion de propiedades.

Los bloques son **independientes en su calificacion**: puedes hacerlos en cualquier orden y obtener puntuacion en uno aunque otro no funcione. Cada conjunto de tests empleara su propia preparacion y archivos de prueba para evitar interferencias. Los ejemplos y contratos de este documento son los que prevalecen para esta evaluacion.

2. Material que recibes

- Proyecto Maven con dependencias y tests preparados.
- **IRepository<T, ID>**: interfaz CRUD **proporcionada**.
- **AbstractFileRepository<T, ID>**: clase abstracta que **implementa IRepository<T, ID>**, con la logica completa de **findAll**, **findById**, **create**, **update** y **delete**.
- Ejemplos de referencia en **demo/crud/**: **CrudDemoCsv**, **CrudDemoJson** y **CrudDemoXml**. **No evaluables**.
- Modelo **Persona** y esqueletos de las clases evaluables.
- Cuatro archivos **preexistentes y poblados** bajo **data/**. No tienes que generar un archivo inicial vacio.

Organizacion prevista

```
filelab/
├── pom.xml
├── data/
│   ├── personas.csv
│   ├── personas.json
│   ├── personas.xml
│   └── academia.properties
└── src/
    └── main/java/es/codelearnacademy/filelab/
```

```

├── repository/
│   ├── IRepository.java           # Entregada, no evaluable
│   └── AbstractFileRepository.java # Entregada, no evaluable
├── demo/crud/
│   ├── CrudDemoCsv.java          # Ejemplo, no evaluable
│   ├── CrudDemoJson.java         # Ejemplo, no evaluable
│   └── CrudDemoXml.java          # Ejemplo, no evaluable
├── model/Persona.java            # Entregada
├── persona/IPersonaRepository.java # Evaluable
├── persona/csv/PersonaCsvRepository.java # Evaluable
├── persona/json/PersonaJsonRepository.java # Evaluable
├── persona/xml/DocumentoPersonas.java # Evaluable
├── persona/xml/PersonaXmlRepository.java # Evaluable
├── config/PropertiesConfig.java    # Evaluable
└── test/java/                    # Pruebas por bloque

```

Los nombres de paquetes de este arbol son la convencion propuesta para la entrega; respeta siempre los paquetes y firmas exactos de los esqueletos proporcionados. **No existe `PersonaService` en esta evaluacion.**

Modelo de datos

```
public record Persona(String dni, String nombre, String email,
                     int edad, boolean activo) { }
```

El identificador unico de una persona es `dni` (`String`). Los tres formatos contienen inicialmente los **mismos cinco registros**:

DNI	Nombre	Email	Edad	Activo
12345678A	Ana Lopez	ana@academia.es	21	true
23456789B	Luis Perez	luis@academia.es	17	true
34567890C	Marta Diaz	marta@academia.es	35	false
45678901D	Carlos Ruiz	carlos@academia.es	29	true
56789012E	Laura Gil	laura@academia.es	16	false

El archivo `academia.properties` contiene al menos:

```
academia.nombre=Centro Formativo
academia.curso=2026
academia.plazas=30
academia.activa=true
datos.formato=csv
datos.ruta=data/personas.csv
```

3. Contratos e interfaces

Contrato comun — se entrega implementado

```
public interface IRepository<T, ID> {  
    List<T> findAll();  
    Optional<T> findById(ID id);  
    boolean create(T entity);  
    boolean update(T entity);  
    boolean delete(ID id);  
}
```

`AbstractFileRepository<T, ID>` **implementa** esta interfaz y aporta el CRUD completo. Define tres operaciones protegidas que especializan los repositorios de ficheros:

```
protected abstract ID getId(T entity);  
protected abstract List<T> readAll() throws IOException;  
protected abstract void writeAll(List<T> entities) throws IOException;
```

Contrato especifico — debe completarlo el alumnado

Crea/completa `IPersonaRepository` para que **extienda** `IRepository<Persona, String>` y declare o implemente como metodos `default` estas **dos consultas**, que deben estar disponibles sobre CSV, JSON y XML:

```
List<Persona> findByEdadMinima(int edad);  
List<Persona> findByActivo(boolean activo);
```

Semantica obligatoria:

- `findByEdadMinima(edad)` devuelve las personas con edad **mayor o igual** a `edad`.
- `findByActivo(true)` devuelve las activas; `findByActivo(false)`, las inactivas.
- Sin coincidencias, se devuelve **lista vacia**, nunca `null`.
- El orden de resultados respeta el orden original de `findAll()`.
- No se altera el archivo al realizar estas consultas.

Recomendacion de implementacion: escribir ambos como metodos `default` en `IPersonaRepository`, apoyandose en `findAll()`; así se programa la consulta una unica vez y la heredan todos los repositorios especificos. Los cuerpos de los metodos seran trabajo evaluable. El docente proporcionara el esqueleto adecuado.

Repositorios especificos — deben completarse

Las **tres clases** siguen el mismo patron; cada una **extiende directamente** `AbstractFileRepository<Persona, String>` e **implementa** `IPersonaRepository`:

```

public class PersonaCsvRepository
    extends AbstractFileRepository<Persona, String>
    implements IPersonaRepository { /* ... */ }

public class PersonaJsonRepository
    extends AbstractFileRepository<Persona, String>
    implements IPersonaRepository { /* ... */ }

public class PersonaXmlRepository
    extends AbstractFileRepository<Persona, String>
    implements IPersonaRepository { /* ... */ }

```

En cada repositorio se completan **getId**, **readAll** y **writeAll** conforme a la firma y a los constructores suministrados. **getId(Persona persona)** devuelve **persona.dni()**. **No** se reprograman **findAll**, **findById**, **create**, **update** ni **delete**.

4. Bloques evaluables y puntuacion

Se dispone de **105 minutos** en total. La distribucion temporal de la tabla es orientativa; puedes organizarte libremente.

Bloque	Entrega	Puntos	Tiempo sugerido
1. IPersonaRepository	Contrato especifico y dos consultas con findAll()	1,5	15 min
2. CSV	PersonaCsvRepository : lectura, escritura, conversion de registros	1,5	15 min
3. JSON	PersonaJsonRepository : serializacion y deserializacion con Jackson	1,5	15 min
4. XML	DocumentoPersonas y PersonaXmlRepository : mapeo y persistencia XML	2,5	25 min
5. Properties	PropertiesConfig : lectura, consulta y persistencia de cambios	2,0	25 min
Subtotal funcional	Pruebas JUnit ponderadas por bloques	9,0	95 min
Documentacion JavaDoc	Documentacion de IPersonaRepository , solo si supera el umbral de tests	1,0	10 min
Total		10,0	105 min

Bloque 1 — **IPersonaRepository** (1,5 puntos)

1. Especializa correctamente **IRepository<Persona, String>**.

2. Implementa `findByEdadMinima(int edad)` y `findByActivo(boolean activo)` (preferentemente como metodos `default`).
3. Comprueba con los cinco registros: `findByEdadMinima(18)` devuelve **Ana, Marta y Carlos**; `findByActivo(true)` devuelve **Ana, Luis y Carlos**.
4. Debe funcionar sin conocer el formato del repositorio y no modificar datos.

Bloque 2 — CSV (1,5 puntos)

1. Completa `PersonaCsvRepository` con las operaciones protegidas de persistencia.
2. **Empieza por las búsquedas:** `findAll()` debe recuperar cinco personas y `findById("12345678A")` debe devolver a Ana.
3. Despues comprueba la escritura: `create` o `update` mediante el CRUD heredado debe persistir los cambios en el archivo CSV.
4. Conserva cabecera y tipos; utiliza Apache Commons CSV y UTF-8. Los campos entrecomillados y con separadores deben interpretarse correctamente.

Bloque 3 — JSON (1,5 puntos)

1. Completa `PersonaJsonRepository` usando **Jackson `ObjectMapper`** y los mecanismos de mapeo adecuados para una lista de `Persona`.
2. **Empieza por las búsquedas:** `findAll()` devuelve cinco personas y `findById("34567890C")` localiza a Marta.
3. Comprueba posteriormente la persistencia de una creacion o eliminacion mediante el CRUD heredado.
4. No construyas JSON manualmente concatenando cadenas.

Bloque 4 — XML (2,5 puntos)

1. Completa `DocumentoPersonas` y sus anotaciones Jackson XML de acuerdo con el esqueleto suministrado.
2. Completa `PersonaXmlRepository`, que **extiende directamente** `AbstractFileRepository<Persona, String>`.
3. El documento debe tener la raiz `<personas>` y elementos repetidos `<persona>`.
4. **Empieza por las búsquedas:** recupera los cinco registros y consulta una persona por DNI.
5. Verifica a continuacion la actualizacion o eliminacion y que los cambios se vuelven a leer desde XML.
6. Utiliza `XmlMapper`; no generes el XML manualmente.

Bloque 5 — Properties (2 puntos, independiente)

Este bloque **no depende de las clases de Persona ni de los repositorios**. Completa en `PropertiesConfig` los metodos:

```
Optional<String> get(String key);
String getOrDefault(String key, String defaultValue);
Map<String, String> findAll();
boolean put(String key, String value);
boolean remove(String key);
```

1. **Empieza por las consultas:** recupera `academia.nombre`, `academia.plazas` y una clave inexistente.
2. `get()` devuelve `Optional.empty()` cuando la clave no existe.
3. `getOrDefault()` devuelve el valor por defecto si falta la clave.
4. `findAll()` devuelve la configuracion actual.
5. `put()` anade o actualiza una clave y **guarda** el cambio en disco.
6. `remove()` elimina una clave y **guarda** el cambio en disco.
7. Los cambios deben mantenerse al crear una nueva instancia de `PropertiesConfig` sobre el mismo archivo.

5. Reglas generales

- **No modificar** `IRepository`, `AbstractFileRepository` ni los `CrudDemo*` proporcionados: su funcionamiento CRUD esta resuelto y **no recibe puntuacion**.
- No cambiar las firmas publicas ni los paquetes de los esqueletos entregados; no editar las pruebas para conseguir una nota artificial.
- No anadir `PersonaService`: toda consulta especifica de personas reside en `IPersonaRepository`.
- Las operaciones CRUD publicas **no propagan** `IOException`; el codigo heredado se encarga de traducir los errores segun el contrato. Los metodos protegidos `readAll()` y `writeAll()` si pueden declararla.
- Como criterio general para fallos de entrada/salida en las API publicas: `Optional.empty()`, listas/mapas vacios o `false`, segun el retorno indicado en cada metodo. En `PropertiesConfig`, se aplica lo definido en su esqueleto.
- Los metodos de consulta no modifican el fichero. Los cambios de los metodos de escritura han de persistir realmente.
- Mantener la codificacion UTF-8 y utilizar las bibliotecas previstas (Commons CSV y Jackson).
- Los cuatro ficheros originales `data/` **ya existen**; los primeros pasos de cada bloque son busquedas o lecturas. **No los sustituyas por archivos vacios**.

6. Pruebas automaticas y archivos de evaluacion

El alumnado puede ejecutar desde la raiz del proyecto:

```
mvn clean verify -Pnota
```

Los tests se organizan por bloques independientes. Se utilizaran los archivos de `data/` para las **consultas iniciales**. Para los tests que modifican datos, el sistema de evaluacion **copia los archivos preexistentes y poblados** a un directorio de trabajo aislado (por ejemplo, `target/test-data/`) y

realiza allí las escrituras. Esto impide que una creación o eliminación cambie la base de datos inicial y altere las pruebas siguientes.

Los archivos con los que empieza el alumno **no son temporales**. Las copias de evaluación solo permiten repetir los tests en las mismas condiciones. Cada bloque se califica por separado, aunque otros bloques estén incompletos.

7. Rubrica y formula de nota

Funcionalidad: hasta 9 puntos

Cada bloque tiene el peso indicado en el apartado 4. Dentro de cada bloque, la parte funcional se calcula con sus tests superados:

$$\text{Nota funcional} = \sum [\text{peso del bloque} \times (\text{tests superados del bloque} / \text{tests del bloque})]$$

El **porcentaje global de tests superados** sirve unicamente para determinar el acceso al punto de documentacion:

$$\text{Porcentaje de tests} = 100 \times (\text{n.º total de tests funcionales superados}) / (\text{n.º total de tests funcionales evaluados})$$

JavaDoc: hasta 1 punto, condicionado

Solo se revisara y puntuara la documentacion si se supera estrictamente el 85 % de los tests funcionales (mas del 85 %, no igual). Si el porcentaje es del 85 % o inferior, la documentacion aporta **0 puntos**, aunque este escrita.

La documentacion evaluable es **exclusivamente la interfaz `IPersonaRepository`**:

Criterio	Puntos
Descripcion de la interfaz y su responsabilidad	0,2
JavaDoc correcto de <code>findByEdadMinima</code>	0,3
JavaDoc correcto de <code>findByActivo</code>	0,3
Uso adecuado de <code>@param</code> , <code>@return</code> y comportamiento sin coincidencias	0,2
Total	1,0

Nota final = nota funcional (maximo 9) + documentacion admitida (maximo 1).

Ejemplos del umbral:

- 34 tests superados de 40 = **85 %**: **no** se evalua JavaDoc.
- 35 tests superados de 40 = **87,5 %**: **si** se evalua JavaDoc.

El porcentaje se calcula sobre todos los tests funcionales; la nota funcional respeta la ponderacion de los apartados, aunque tengan numeros distintos de tests.

8. Entrega y recomendaciones

- Entrega el proyecto Maven con las clases evaluables completadas, siguiendo el procedimiento indicado por el profesor.
- No incluyas archivos generados dentro de `target/` ni modifiques `pom.xml` salvo autorizacion expresa.
- Comprueba primero los datos iniciales con `findAll()` y `findById()`; despues prueba las escrituras.
- Si una actividad se complica, continua con otra: **cada bloque es independiente.**
- Reserva unos minutos para comprobar los tests y la documentacion de `IPersonaRepository`.

Se valorara la correccion del resultado, la reutilizacion de la arquitectura suministrada y el respeto a los contratos establecidos; no se calificara la reimplementacion de los CRUD proporcionados.

Ejecucion de la autoevaluacion Maven

Desde la raiz del proyecto, con **JDK 21, Maven y Python 3** instalados:

```
mvn clean verify -Pnota
```

Este comando limpia informes antiguos, compila, ejecuta JUnit y, en la fase `verify`, ejecuta el calculo automatico de los **9 puntos funcionales**. El perfil `nota` permite mostrar el informe aun cuando fallen tests; **que Maven termine sin error no significa que se hayan superado todas las pruebas**. Revisa el resumen de JUnit y la puntuacion impresa.

El **punto de JavaDoc se corrige automaticamente** mediante `mvn clean verify -Pnota`, y solo se considera con mas del 85 % de tests superados.

Politica de excepciones

Ninguna operacion declarada en `IRepository` o `IPersonaRepository` propaga `IOException`: los metodos CRUD publicos devuelven listas vacias, `Optional.empty()` o `false` segun su contrato si falla el acceso al fichero. Los metodos **protegidos** `readAll()` y `writeAll()` de `AbstractFileRepository` pueden declarar `IOException` porque son puntos de extension internos; la clase abstracta captura esa excepcion antes de devolver un resultado al codigo cliente.

Evaluacion automatica de JavaDoc (1 punto)

El comando `mvn clean verify -Pnota` calcula la nota funcional (9 puntos) y **corrige automaticamente el JavaDoc de `IPersonaRepository` (1 punto)**. La documentacion **solo puntua si se supera estrictamente el 85 % de los tests funcionales** y estan presentes los informes de los cinco bloques. Un 85,00 % exacto no da acceso al punto.

Se aceptan expresiones equivalentes, sin distinguir mayusculas. Solo se utilizan palabras clave sin acentos. La documentacion debe ser un comentario `/** ... */` inmediatamente anterior a cada

declaracion, suficientemente descriptivo y sin marcadores **TODO** o **pendiente**.

Criterio	Puntos	Palabras/elementos requeridos
Interfaz <code>IPersonaRepository</code>	0,2	<code>persona</code> o <code>personas</code> , y <code>repositorio</code> o <code>consulta/consultas</code>
<code>findByEdadMinima</code>	0,3	<code>edad</code> , <code>minima</code> o <code>mayor</code> o <code>igual</code> / <code>al menos</code> / <code>igual</code> o <code>superior</code> , <code>@param edad</code> , <code>@return</code>
<code>findByActivo</code>	0,3	<code>activo</code> / <code>activa</code> / <code>actividad</code> / <code>estado</code> , referencia a valores <code>true</code> y <code>false</code> o a <code>activo/inactivo</code> , <code>@param activo</code> , <code>@return</code>
Comportamiento sin resultados	0,2	En ambos metodos: <code>lista vacia</code> / <code>vacía</code> / <code>sin coincidencias</code> / <code>sin resultados</code> / <code>no hay coincidencias</code> , además de <code>@return</code>

La puntuacion documental puede obtenerse parcialmente si se supera el umbral funcional. La correccion es una **comprobacion textual objetiva**, no una valoracion semantica completa: el docente podra revisar manualmente documentacion que este bien redactada con sinonimos no reconocidos o comentarios enganosos que cumplan artificialmente las palabras clave.

Ejemplo valido de JavaDoc

```
/** Repositorio de personas que permite consultas especificas. */
public interface IPersonaRepository extends IRepository<Persona, String> {

    /**
     * Busca personas con edad mayor o igual a la edad minima indicada.
     *
     * @param edad edad minima inclusiva
     * @return lista de personas coincidentes; lista vacia si no hay
    resultados
     */
    default List<Persona> findByEdadMinima(int edad) {
        // Implementacion pendiente en la version del alumnado
    }

    /**
     * Consulta personas segun su estado activo o inactivo.
     *
     * @param activo true para activas y false para inactivas
     * @return lista de personas coincidentes; vacia si no hay
    coincidencias
     */
    default List<Persona> findByActivo(boolean activo) {
        // Implementacion pendiente en la version del alumnado
    }
}
```

Entrega

Trabaja en las clases evaluables indicadas en `README.md`. Ejecuta `mvn clean verify -Pnota```. Los `archivos` `data/`` contienen informacion inicial. No modifiques la infraestructura CRUD.